Beyond Conventional Mathematics

An Experimental Journey into AI-Discovered Mathematical Systems

Version 1.0 (research batch 1, cycles C01-C20) — built 2026-10-18T07:43:58Z

- Preface
- 1 How the laboratory works
  - 1.1 Minimal assumptions, stated twice
  - 1.2 A research cycle
  - 1.3 Kinds of proof
  - 1.4 Novelty

Preface

This book is written by a research laboratory rather than by a single author. The laboratory — a small team of cooperating AI research roles (an Inventor, an Explorer, a Formal Verifier, a Skeptic, an Application Researcher, a Historian and an Author) — set out to explore mathematical systems built from deliberately minimal assumptions: distinguishable objects, equality, and explicitly stated transformation rules. Numbers, positional notation, multiplication and even “value” were not assumed; where they appear in these pages, an earlier chapter shows how they *emerge*.

The book has one rule that overrides every other consideration:

> **Nothing is presented as established unless it has been proved, and nothing is presented as new unless we looked for it and failed to find it.**

Every theorem, conjecture and refuted hypothesis carries a label:

Label	Meaning
PROVEN (Lean)	machine-checked in Lean 4 with Mathlib, standard axioms only
PROVEN (computer-assisted)	a finite certificate checked by a program; the trust base is stated
PROVEN (written)	a complete written proof, reviewed by the lab’s independent Skeptic
COMPUTATIONALLY SUPPORTED	experiments support the claim; no proof
CONJECTURE	a precise claim awaiting proof
REFUTED	a counterexample is given — and kept, because failures teach
REDISCOVERY	the result is already in the literature (references given)
POTENTIALLY NOVEL	no equivalent found in the sources searched; *not* a priority claim

Most of what the laboratory found is, honestly, rediscovery: the commutative monoid of counts, chip-firing, rational base numeration, the Zeckendorf game, Knuth’s Fibonacci multiplication. We regard this as a success of the method — independent rediscovery from minimal assumptions is evidence that the structures are natural — and we document the literature carefully. Where something appears to be new (for

example, the common-border formula for words modulo one relation, or the density of irreducibles for Knuth’s circle product), we say exactly how far our search went.

Each chapter follows the same plan: motivation, definitions, examples, results with proofs or proof attempts, the computational experiments behind them (with experiment identifiers that can be re-run from the repository), counterexamples, applications, comparison with existing mathematics, limitations, and open questions.

How to verify anything in this book. Every experiment identifier (EXP-C01-05, …) names a deterministic script in the repository; `python -m lab run EXP-C01-05` re-runs it and checks the recorded result hash. Every claim identifier (CLM-C08-01, …) has a record in `db/claims/` with its full status history. Lean theorem names refer to `proofs/lean/Infinity/`.

1 How the laboratory works

1.1 Minimal assumptions, stated twice

Each research branch begins from a *foundational system* whose object-level axioms are written down explicitly (repository file `research/systems/SYSTEMS.md`):

- **System A** — words over distinguishable symbols, juxtaposition, rewriting rules; no numbers.
- **System B** — a carrier with one invented binary operation; no algebraic law is assumed, every law is a hypothesis to be tested.
- **System C** — representations of quantities (digit strings with a valuation), admissible only after System A has *derived* counting.
- **System D** — structures defined by relations only (game options, graphs).
- **System E** — local-rule dynamics over invented algebras.

The *meta level* — the mathematics we use to prove things about these systems — is ordinary mathematics (formally, Lean's type theory with Mathlib). When we say that a concept "emerges", we mean that it is definable from the system's primitives and forced by them up to isomorphism, as shown by a meta-level proof.

1.2 A research cycle

Every cycle runs ten steps, and the cycle report in the repository is organised by them:

1. generate hypotheses — including at least one *bold* hypothesis that might fail;
2. state them precisely;
3. design experiments that could falsify them;
4. implement the experiments (deterministic, seeded scripts);
5. execute them (results are hashed; re-running checks the hash);
6. analyse the results;
7. attempt proofs or find counterexamples;
8. compare with the literature (OEIS and web searches; references recorded);
9. look for applications, with baselines and honest negatives;
10. document everything in the journal and the database.

1.3 Kinds of proof

Lean proofs. Statements are formalised in Lean 4 with Mathlib; a file counts as verified only if it compiles with no `sorry` and `#print axioms` reports only `propext`,

`Classical.choice` and `Quot.sound`.

Computer-assisted proofs. Some results are proved by finite certificates that a program checks: a completed rewriting system whose convergence is re-verified independently, an automaton whose count sequence obeys a recurrence of bounded order, a recurring window of a monotone map. For these the book states the *trust base* — the program code — and the cross-checks used to guard against implementation errors.

Written proofs are complete arguments reviewed by the lab's Skeptic, who works independently from the authors and tries to break every claim.

1.4 Novelty

The Historian searches the OEIS (all integer sequences we produce are looked up, and the cached answers are kept in the repository) and the literature. Matches are *read*, not counted: in Chapter 9 a sequence agreed with an OEIS entry for 24 terms and then diverged; the agreement had a simple explanation and was recorded as a coincidence.